МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

# Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Новак Дмитрий Евгеньевич
Группа: М8О-201БВ-25
Преподаватель: Миронов Евгений Сергеевич
Дата: 06.10.2026
Оценка: _____

Москва, 2026

## Условие

**Цель работы:** изучить управление процессами в ОС семейства UNIX и организацию межпроцессного взаимодействия с помощью неименованных каналов (pipe), а также получить практические навыки работы с системными вызовами создания, запуска и ожидания процессов, перенаправления стандартных потоков и обработки системных ошибок.

**Задание:** родительский процесс создаёт дочерний процесс. Пользователь задаёт путь к входному файлу; файл открывается на чтение и становится стандартным вводом дочернего процесса. Стандартный вывод дочернего процесса направляется в канал. Родительский процесс читает данные из канала и выводит их в свой стандартный поток вывода. Родительская и дочерняя части реализуются отдельными программами. Ошибки системных вызовов должны обрабатываться.

**Вариант 9:** каждая строка входного файла содержит произвольное количество чисел типа `float`. Дочерний процесс делит первое число строки последовательно на все следующие числа и выводит результат. Проверка деления на ноль выполняется в дочернем процессе; при обнаружении нулевого делителя выполнение завершается с ошибкой.

## Метод решения

Программа состоит из двух исполняемых файлов: `lab2_parent` и `lab2_child`. Родитель сначала считывает введённый пользователем путь и открывает соответствующий файл только для чтения. Затем создаётся неименованный канал и дочерний процесс.

После `fork()` дочерний процесс перенаправляет открытый файл на свой `stdin` с помощью `dup2()`, а записывающий конец канала — на свой `stdout`. После закрытия лишних дескрипторов выполняется `execl()`, заменяющий код дочернего процесса программой `lab2_child`. Дочерняя программа читает стандартный ввод и пишет в стандартный вывод. Источник ввода и получатель вывода определяются перенаправлением дескрипторов.

Дочерняя программа читает файл построчно функцией `getline()`, преобразует числа с помощью `strtof()` и последовательно выполняет деление. Нулевой делитель проверяется до операции деления. Результат каждой строки выводится в `stdout`, поэтому фактически попадает в канал.

Родитель закрывает неиспользуемый записывающий конец канала и в цикле читает результаты из читающего конца. Полученные байты записываются в стандартный вывод родителя. После получения EOF из канала родитель закрывает дескриптор и ожидает завершения дочернего процесса с помощью `waitpid()`, после чего анализирует код его завершения.

# Описание программы

## Структура программной части

Таблица 1 — Файлы программной части

| Файл | Назначение |
|---|---|
| `src/parent.c` | Родительский процесс: получает путь к файлу, открывает его, создаёт канал и дочерний процесс, получает результаты из канала и ожидает завершения ребёнка. |
| `src/child.c` | Дочерняя программа: читает строки из стандартного ввода, выполняет деление первого числа на последующие и выводит результат. |
| `CMakeLists.txt` | Настройка сборки двух исполняемых файлов `lab2_parent` и `lab2_child`. |
| `data/*.txt` | Тестовые входные файлы для корректного ввода, отрицательных чисел и деления на ноль. |

Исходный код в листингах подключается напрямую из `src/`, без копий в каталоге отчёта. Трассировка разобрана в отдельном разделе и приложении Б.

### Основные типы данных и функции

В родительском процессе путь к входному файлу считывается функцией `getline()` в динамически выделяемый буфер `filename`. После успешного `open()` строка пути больше не требуется и освобождается функцией `free()`. Дескриптор входного файла хранится в `input_fd`, а два конца канала — в массиве `pipe_fd[2]`.

Для передачи данных родитель использует буфер `char buffer[4096]`. Дочерний процесс хранит текущую строку в динамическом буфере `line`, который переиспользуется между вызовами `getline()` и освобождается перед выходом.

В `child.c` вспомогательная функция `read_float(char* text, char** end, float* value)` преобразует очередное число через `strtof()`, проверяет отсутствие преобразования, ошибку `ERANGE` и нечисловые/бесконечные значения. Возвращаемое значение 1 означает успешное чтение числа, 0 — ошибку.

Программа использует отдельные коды завершения: 1 — общая ошибка, 2 — деление на ноль, 3 — некорректные входные данные, 4 — ошибка запуска дочерней программы через `exec`.

### Используемые системные вызовы

Таблица 2 — Используемые системные вызовы и POSIX-интерфейсы

| Вызов | Назначение в программе |
|---|---|
| `open(path, O_RDONLY)` | Открытие указанного пользователем входного файла только для чтения. |
| `pipe(pipe_fd)` | Создание однонаправленного канала между дочерним и родительским процессами. |
| `fork()` | Создание дочернего процесса. |
| `dup2(oldfd, newfd)` | Перенаправление стандартного ввода ребёнка на файл и стандартного вывода ребёнка в канал. |
| `execl()` | Замена кода дочернего процесса программой `lab2_child`. |
| `read()` | Чтение родителем результатов из канала. |
| `write()` | Вывод родителем полученных из канала байтов в `stdout`. |
| `close()` | Закрытие неиспользуемых и более не нужных файловых дескрипторов. |
| `waitpid()` | Ожидание завершения конкретного дочернего процесса и получение его статуса. |

### Ключевые фрагменты кода

Сначала родитель открывает заданный пользователем файл, создаёт канал и дочерний процесс. Ошибки каждого системного вызова проверяются.

Листинг 1 — Открытие файла, создание канала и процесса (parent.c)

```
    int input_fd = open(filename, O_RDONLY);

    if (input_fd == -1) {
        perror("open");
        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    free(filename);

    int pipe_fd[2];

    if (pipe(pipe_fd) == -1) {
        perror("pipe");

        if (close(input_fd) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    pid_t pid = fork();
```

```
    if (pid == -1) {
        perror("fork");

        if (close(input_fd) == -1) {
            perror("close");
        }

        if (close(pipe_fd[0]) == -1) {
            perror("close");
        }

        if (close(pipe_fd[1]) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }
```

В дочерней ветви закрывается читающий конец канала. Затем входной файл дублируется в дескриптор 0, а записывающий конец канала — в дескриптор 1. После этого `execl()` запускает отдельную дочернюю программу.

Листинг 2 — Перенаправление стандартных потоков и запуск child (parent.c)

```
    if (pid == 0) {
        if (close(pipe_fd[0]) == -1) {
            perror("close");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        if (dup2(input_fd, STDIN_FILENO) == -1) {
            perror("dup2");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        if (dup2(pipe_fd[1], STDOUT_FILENO) == -1) {
            perror("dup2");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        if (input_fd != STDIN_FILENO && close(input_fd) == -1) {
            perror("close");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        if (pipe_fd[1] != STDOUT_FILENO && close(pipe_fd[1]) == -1) {
            perror("close");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        execl("./lab2_child", "lab2_child", (char*)NULL);
```

```
        perror("execl");
        exit(EXEC_ERROR_EXIT_CODE);
    }
```

Родитель читает поток байтов из канала до EOF и передаёт полученные данные в свой стандартный вывод.

Листинг 3 — Чтение результатов из канала (parent.c)

```
    int parent_error = 0;

    if (close(input_fd) == -1) {
        perror("close");
        parent_error = 1;
    }

    if (close(pipe_fd[1]) == -1) {
        perror("close");
        parent_error = 1;
    }

    char buffer[4096];

    while (!parent_error) {
        ssize_t bytes_read = read(pipe_fd[0], buffer, sizeof(buffer));

        if (bytes_read > 0) {
            ssize_t bytes_written = write(STDOUT_FILENO, buffer, (size_t)
    bytes_read);

            if (bytes_written == -1) {
                if (errno == EINTR) {
                    continue;
                }

                perror("write");
                parent_error = 1;
                break;
            }

            if (bytes_written != bytes_read) {
                fprintf(stderr, "Ошибка: данные выведены не полностью.\n");
                parent_error = 1;
                break;
            }

            continue;
        }

        if (bytes_read == 0) {
            break;
        }
```

```
        if (errno == EINTR) {
            continue;
        }

        perror("read");
        parent_error = 1;
        break;
    }

    if (close(pipe_fd[0]) == -1) {
        perror("close");
        parent_error = 1;
    }
```

Дочерняя программа проверяет нулевой делитель до выполнения операции деления.

Листинг 4 — Проверка деления на ноль (child.c)

```
            float divisor;

            if (!read_float(current, &end, &divisor)) {
                fprintf(stderr, "Ошибка в строке %zu: некорректное число.\n",
    line_number);
                free(line);
                return INPUT_ERROR_EXIT_CODE;
            }

            if (divisor == 0.0f) {
                fprintf(stderr, "Ошибка в строке %zu: деление на ноль.\n",
    line_number);
                free(line);
                return DIVISION_BY_ZERO_EXIT_CODE;
            }

            result /= divisor;

            if (!isfinite(result)) {
                fprintf(stderr, "Ошибка в строке %zu: результат вычисления
    вышел за диапазон float.\n", line_number);
                free(line);
                return INPUT_ERROR_EXIT_CODE;
            }

            current = end;
```

## Результаты

Проект собирался CMake с включёнными предупреждениями `-Wall -Wextra`. После сборки оба исполняемых файла находятся в одной директории `build/lab_02`. Команды запуска ниже выполняются из этой директории: путь к входному файлу пользователь вводит сам, а `./lab2_child` ищется относительно текущей рабочей директории.

Пример успешного запуска на тестовом файле `ok.txt`. После приглашения «Введите путь к файлу:» пользователь вводит указанный путь:

```
$ ./lab2_parent
../../lab_02/data/ok.txt
0.5
20
10
100000
500
0.05
0
```

Дополнительно были проверены файлы `negatives.txt` с отрицательными числами и `zero.txt` с нулевым делителем. Отрицательные значения корректно обрабатываются как значения типа `float`. При появлении нуля среди делителей дочерний процесс выводит сообщение об ошибке в `stderr` и завершается с кодом 2; родитель получает этот статус через `waitpid()` и завершает работу с тем же кодом. При проверочном запуске `zero.txt` сообщение в `stderr` имеет вид `Ошибка в строке 1: деление на ноль.`

Проверено перенаправление стандартных потоков оболочкой. Следующая команда скрывает обычный вывод и оставляет видимыми только сообщения об ошибках:

```
$ echo "../../lab_02/data/zero.txt" | ./lab2_parent > /dev/null
```

А команда ниже, наоборот, скрывает `stderr` и оставляет `stdout`:

```
$ echo "../../lab_02/data/zero.txt" | ./lab2_parent 2> /dev/null
```

Потоки также можно сохранить в разные файлы:

```
$ echo "../../lab_02/data/zero.txt" | ./lab2_parent > results.txt 2> errors.txt
```

При таком запуске с `zero.txt` код завершения равен 2, в `errors.txt` находится сообщение о делении на ноль, а в `results.txt` — только приглашение ввести путь.

## Демонстрация работы strace

Трассировка выполнена в Ubuntu ARM64 командой `strace -f`; параметр `-f` необходим, чтобы продолжить трассировку после `fork()` и видеть как родительский, так и дочерний процесс. Ниже приведены фрагменты файла `trace/strace_short.txt`. Полная трассировка аналогичного запуска `trace/strace.txt` приведена в приложении Б; она получена отдельно, поэтому PID процессов в двух файлах различаются.

Сначала родитель считывает путь, открывает файл, создаёт канал и дочерний процесс:

```
320721 read(0, "../../lab_02/data/ok.txt\n", 4096) = 25
320721 openat(AT_FDCWD, "../../lab_02/data/ok.txt", O_RDONLY) = 3
320721 pipe2([4, 5], 0)                 = 0
320721 clone(child_stack=NULL,
↪  flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↪  child_tidptr=0xe73562848030) = 320722
```

В исходном коде используются `open()`, `pipe()` и `fork()`, однако `strace` показывает фактические системные вызовы Linux. В данной системе соответствующие обёртки libc реализованы через `openat()`, `pipe2()` и `clone()`. В результате файл получил дескриптор 3, читающий конец канала — 4, записывающий — 5; PID ребёнка равен 320722.

Далее дочерний процесс перенаправляет дескрипторы и запускает `lab2_child`:

```
320722 dup3(3, 0, 0 <unfinished ...>
320721 read(4 <unfinished ...>
320722 <... dup3 resumed>)              = 0
320722 dup3(5, 1, 0)                    = 1
320722 close(3)                         = 0
320722 close(5)                         = 0
320722 execve("./lab2_child", ["lab2_child"], 0xfffff7568088 /* 61 vars */) = 0
```

В программе вызван `dup2()`, но на Ubuntu ARM64 libc использует системный вызов `dup3()` с нулевыми флагами. После первого вызова дескриптор 0 (`stdin`) указывает на входной файл, после второго дескриптор 1 (`stdout`) — на записывающий конец канала. Вызов `execl()` в исходном коде приводит к системному вызову `execve()` и не создаёт новый процесс: PID 320722 сохраняется.

Передача результата через канал видна непосредственно в трассировке:

```
320722 write(1, "0.5\n", 4)             = 4
320722 write(1, "20\n", 3)              = 3
320722 write(1, "10\n", 3)              = 3
320722 write(1, "100000\n", 7)          = 7
320722 write(1, "500\n", 4)             = 4
320721 <... read resumed>, "0.5\n20\n10\n100000\n500\n", 4096) = 21
320722 write(1, "0.05\n", 5)            = 5
320721 write(1, "0.5\n20\n10\n100000\n500\n", 21 <unfinished ...>
320722 read(0 <unfinished ...>
320721 <... write resumed>)             = 21
```

Дочерний процесс пишет результаты в свой `stdout`, который направлен в канал. Родитель читает те же байты из дескриптора 4 и выводит их на экран. При этом несколько отдельных операций `write()` ребёнка могут быть получены одним `read()` родителя, поскольку pipe представляет собой поток байтов, а не набор сообщений.

После завершения ребёнка родитель получает EOF из канала, закрывает его и ожидает дочерний процесс:

```
320721 write(1, "0.05\n0\n", 7)         = 7
320721 read(4, "", 4096)                = 0
320721 close(4)                         = 0
320721 wait4(320722, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 320722
320721 +++ exited with 0 +++
```

В исходном коде используется `waitpid()`, а трассировка Linux показывает `wait4()`. Ребёнок завершился с кодом 0; затем с тем же кодом завершился родитель.

## Выводы

Таким образом, данная лабораторная работа никак кардинально не изменила мою жизнь, но позволила более подробно изучить работу процессов и их взаимодействие через каналы, а также узнать, что на самом деле даже самые большие программы являются fork-ами от чего-то другого.

## Приложение А. Исходный код программы

Листинг А.1 — Файл parent.c

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define GENERAL_ERROR_EXIT_CODE 1
#define DIVISION_BY_ZERO_EXIT_CODE 2
#define INPUT_ERROR_EXIT_CODE 3
#define EXEC_ERROR_EXIT_CODE 4

int main(void) {
    char* filename = NULL;
    size_t capacity = 0;

    printf("Введите путь к файлу: ");

    if (fflush(stdout) == EOF) {
        perror("fflush");
        return GENERAL_ERROR_EXIT_CODE;
    }

    ssize_t length = getline(&filename, &capacity, stdin);

    if (length == -1) {
        if (ferror(stdin)) {
            perror("getline");
        } else {
            fprintf(stderr, "\nОшибка: путь к файлу не введен.\n");
        }

        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    while (length > 0 && filename[length - 1] == '\n') {
        filename[--length] = '\0';
    }

    if (length == 0) {
        fprintf(stderr, "Ошибка: путь к файлу не может быть пустым.\n");
        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    int input_fd = open(filename, O_RDONLY);

    if (input_fd == -1) {
        perror("open");
        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    free(filename);

    int pipe_fd[2];

    if (pipe(pipe_fd) == -1) {
        perror("pipe");
```

```
    if (close(input_fd) == -1) {
        perror("close");
    }

    return GENERAL_ERROR_EXIT_CODE;
}

pid_t pid = fork();

if (pid == -1) {
    perror("fork");

    if (close(input_fd) == -1) {
        perror("close");
    }

    if (close(pipe_fd[0]) == -1) {
        perror("close");
    }

    if (close(pipe_fd[1]) == -1) {
        perror("close");
    }

    return GENERAL_ERROR_EXIT_CODE;
}

if (pid == 0) {
    if (close(pipe_fd[0]) == -1) {
        perror("close");
        exit(GENERAL_ERROR_EXIT_CODE);
    }

    if (dup2(input_fd, STDIN_FILENO) == -1) {
        perror("dup2");
        exit(GENERAL_ERROR_EXIT_CODE);
    }

    if (dup2(pipe_fd[1], STDOUT_FILENO) == -1) {
        perror("dup2");
        exit(GENERAL_ERROR_EXIT_CODE);
    }

    if (input_fd != STDIN_FILENO && close(input_fd) == -1) {
        perror("close");
        exit(GENERAL_ERROR_EXIT_CODE);
    }

    if (pipe_fd[1] != STDOUT_FILENO && close(pipe_fd[1]) == -1) {
        perror("close");
        exit(GENERAL_ERROR_EXIT_CODE);
    }

    execl("./lab2_child", "lab2_child", (char*)NULL);

    perror("execl");
    exit(EXEC_ERROR_EXIT_CODE);
}

int parent_error = 0;

if (close(input_fd) == -1) {
    perror("close");
```

```
    parent_error = 1;
}

if (close(pipe_fd[1]) == -1) {
    perror("close");
    parent_error = 1;
}

char buffer[4096];

while (!parent_error) {
    ssize_t bytes_read = read(pipe_fd[0], buffer, sizeof(buffer));

    if (bytes_read > 0) {
        ssize_t bytes_written = write(STDOUT_FILENO, buffer, (size_t)bytes_read);

        if (bytes_written == -1) {
            if (errno == EINTR) {
                continue;
            }

            perror("write");
            parent_error = 1;
            break;
        }

        if (bytes_written != bytes_read) {
            fprintf(stderr, "Ошибка: данные выведены не полностью.\n");
            parent_error = 1;
            break;
        }

        continue;
    }

    if (bytes_read == 0) {
        break;
    }

    if (errno == EINTR) {
        continue;
    }

    perror("read");
    parent_error = 1;
    break;
}

if (close(pipe_fd[0]) == -1) {
    perror("close");
    parent_error = 1;
}

int status;

while (waitpid(pid, &status, 0) == -1) {
    if (errno == EINTR) {
        continue;
    }

    perror("waitpid");
    return GENERAL_ERROR_EXIT_CODE;
}
```

```
    if (parent_error) {
        return GENERAL_ERROR_EXIT_CODE;
    }

    if (WIFEXITED(status)) {
        int child_status = WEXITSTATUS(status);

        if (child_status == 0) {
            return 0;
        }

        if (child_status == DIVISION_BY_ZERO_EXIT_CODE) {
            return DIVISION_BY_ZERO_EXIT_CODE;
        }

        if (child_status == INPUT_ERROR_EXIT_CODE) {
            return INPUT_ERROR_EXIT_CODE;
        }

        if (child_status == EXEC_ERROR_EXIT_CODE) {
            fprintf(stderr, "Ошибка: не удалось запустить дочерний процесс.\n");
            return EXEC_ERROR_EXIT_CODE;
        }

        fprintf(stderr, "Дочерний процесс завершился с кодом %d.\n", child_status);
        return child_status;
    }

    if (WIFSIGNALED(status)) {
        fprintf(stderr, "Дочерний процесс завершён сигналом %d.\n", WTERMSIG(status))
    ;
        return GENERAL_ERROR_EXIT_CODE;
    }

    fprintf(stderr, "Ошибка: дочерний процесс завершился в неизвестном состоянии.\n")
    ;
    return GENERAL_ERROR_EXIT_CODE;
}
```

Листинг А.2 — Файл child.c

```
#include <ctype.h>
#include <errno.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

#define GENERAL_ERROR_EXIT_CODE 1
#define DIVISION_BY_ZERO_EXIT_CODE 2
#define INPUT_ERROR_EXIT_CODE 3

int read_float(char* text, char** end, float* value) {
    errno = 0;
    *value = strtof(text, end);

    if (text == *end) {
        return 0;
    }

    if (errno == ERANGE || !isfinite(*value)) {
        return 0;
    }

    return 1;
}
```

```
int main(void) {
    char* line = NULL;
    size_t capacity = 0;
    size_t line_number = 0;

    while (getline(&line, &capacity, stdin) != -1) {
        line_number++;

        char* current = line;

        while (isspace((unsigned char)*current)) {
            current++;
        }

        if (*current == '\0') {
            continue;
        }

        char* end = NULL;
        float result;

        if (!read_float(current, &end, &result)) {
            fprintf(stderr, "Ошибка в строке %zu: некорректное число.\n", line_number
    );
            free(line);
            return INPUT_ERROR_EXIT_CODE;
        }

        current = end;

        while (1) {
            while (isspace((unsigned char)*current)) {
                current++;
            }

            if (*current == '\0') {
                break;
            }

            float divisor;

            if (!read_float(current, &end, &divisor)) {
                fprintf(stderr, "Ошибка в строке %zu: некорректное число.\n",
    line_number);
                free(line);
                return INPUT_ERROR_EXIT_CODE;
            }

            if (divisor == 0.0f) {
                fprintf(stderr, "Ошибка в строке %zu: деление на ноль.\n",
    line_number);
                free(line);
                return DIVISION_BY_ZERO_EXIT_CODE;
            }

            result /= divisor;

            if (!isfinite(result)) {
                fprintf(stderr, "Ошибка в строке %zu: результат вычисления вышел за
    диапазон float.\n", line_number);
                free(line);
                return INPUT_ERROR_EXIT_CODE;
            }
```

```
            current = end;
        }

        if (printf("%g\n", (double)result) < 0) {
            perror("printf");
            free(line);
            return GENERAL_ERROR_EXIT_CODE;
        }

        if (fflush(stdout) == EOF) {
            perror("fflush");
            free(line);
            return GENERAL_ERROR_EXIT_CODE;
        }
    }

    if (ferror(stdin)) {
        perror("getline");
        free(line);
        return GENERAL_ERROR_EXIT_CODE;
    }

    free(line);
    return 0;
}
```

Листинг A.3 — Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(lab_02_variant_9 C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS ON)

add_compile_options(-Wall -Wextra)

add_executable(lab2_parent
    src/parent.c
)

add_executable(lab2_child
    src/child.c
)
```

## Приложение Б. Полный вывод strace

Полный вывод получен при запуске программы на Ubuntu ARM64 с параметром `-f`, позволяющим трассировать дочерний процесс.

Листинг Б.1 — Полный вывод strace -f ./lab2_parent

```
318709 execve("./lab2_parent", ["./lab2_parent"], 0xffffe9366148 /* 61 vars */) = 0
318709 brk(NULL)                        = 0xb1a07ea47000
318709 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf5f89a04b000
318709 faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
↪  directory)
318709 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
318709 fstat(3, {st_mode=S_IFREG|0644, st_size=63707, ...}) = 0
318709 mmap(NULL, 63707, PROT_READ, MAP_PRIVATE, 3, 0) = 0xf5f89a005000
318709 close(3)                         = 0
318709 openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC)
↪  = 3
318709 read(3,
↪  "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\0\0"..., 832) =
↪  832
318709 fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
318709 mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1,
↪  0) = 0xf5f899e27000
318709 mmap(0xf5f899e30000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf5f899e30000
318709 munmap(0xf5f899e27000, 36864)    = 0
318709 munmap(0xf5f899ffe000, 28224)    = 0
318709 mprotect(0xf5f899fd9000, 81920, PROT_NONE) = 0
318709 mmap(0xf5f899fed000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000) = 0xf5f899fed000
318709 mmap(0xf5f899ff2000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf5f899ff2000
318709 mmap(0xf5f899e30000, 1740800, PROT_READ|PROT_EXEC|0x10,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf5f899e30000
318709 close(3)                         = 0
318709 set_tid_address(0xf5f89a04c388)  = 318709
318709 set_robust_list(0xf5f89a04c040, 24) = 0
318709 rseq(0xf5f89a04c740, 0x21, 0, 0xd428bc00) = 0
318709 mprotect(0xf5f899fed000, 12288, PROT_READ) = 0
318709 mprotect(0xb1a06750f000, 4096, PROT_READ) = 0
318709 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf5f89a049000
318709 mprotect(0xf5f89a053000, 8192, PROT_READ) = 0
318709 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↪  rlim_max=RLIM64_INFINITY}) = 0
318709 getrandom("\xf1\x32\x7e\x3c\x06\xec\x2c\x77", 8, GRND_NONBLOCK) = 8
318709 munmap(0xf5f89a005000, 63707)    = 0
318709 fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x1), ...}) = 0
318709 brk(NULL)                        = 0xb1a07ea47000
318709 brk(0xb1a07ec00000)              = 0xb1a07ec00000
318709 write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265
↪  \320\277\321\203\321\202\321\214 \320\272 \321\204\320\260\320"..., 39) = 39
318709 fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
318709 read(0, "../../lab_02/data/ok.txt\n", 4096) = 25
318709 openat(AT_FDCWD, "../../lab_02/data/ok.txt", O_RDONLY) = 3
318709 pipe2([4, 5], 0)                 = 0
318709 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
318709 clone(child_stack=NULL,
↪  flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD <unfinished ...>
318710 set_robust_list(0xf5f89a04c040, 24) = 0
```

```
318710 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
318709 <... clone resumed>, child_tidptr=0xf5f89a04c030) = 318710
318710 close(4 <unfinished ...>
318709 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
318710 <... close resumed>)             = 0
318709 close(3)                         = 0
318710 dup3(3, 0, 0 <unfinished ...>
318709 close(5)                         = 0
318710 <... dup3 resumed>)              = 0
318709 read(4 <unfinished ...>
318710 dup3(5, 1, 0)                    = 1
318710 close(3)                         = 0
318710 close(5)                         = 0
318710 execve("./lab2_child", ["lab2_child"], 0xffffff4cffc48 /* 61 vars */) = 0
318710 brk(NULL)                        = 0xc231b3239000
318710 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf54be0e43000
318710 faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
↪  directory)
318710 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
318710 fstat(3, {st_mode=S_IFREG|0644, st_size=63707, ...}) = 0
318710 mmap(NULL, 63707, PROT_READ, MAP_PRIVATE, 3, 0) = 0xf54be0dfd000
318710 close(3)                         = 0
318710 openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC)
↪  = 3
318710 read(3,
↪  "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\0\0\0\0\0"..., 832) =
↪  832
318710 fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
318710 mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1,
↪  0) = 0xf54be0c1f000
318710 mmap(0xf54be0c20000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf54be0c20000
318710 munmap(0xf54be0c1f000, 4096)     = 0
318710 munmap(0xf54be0dee000, 60992)    = 0
318710 mprotect(0xf54be0dc9000, 81920, PROT_NONE) = 0
318710 mmap(0xf54be0ddd000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000) = 0xf54be0ddd000
318710 mmap(0xf54be0de2000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf54be0de2000
318710 mmap(0xf54be0c20000, 1740800, PROT_READ|PROT_EXEC|0x10,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf54be0c20000
318710 close(3)                         = 0
318710 set_tid_address(0xf54be0e44388)  = 318710
318710 set_robust_list(0xf54be0e44040, 24) = 0
318710 rseq(0xf54be0e44740, 0x21, 0, 0xd428bc00) = 0
318710 mprotect(0xf54be0ddd000, 12288, PROT_READ) = 0
318710 mprotect(0xc2319f87f000, 4096, PROT_READ) = 0
318710 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf54be0e41000
318710 mprotect(0xf54be0e4b000, 8192, PROT_READ) = 0
318710 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↪  rlim_max=RLIM64_INFINITY}) = 0
318710 getrandom("\x71\x8f\xe5\x07\xac\x62\x8d\xfb", 8, GRND_NONBLOCK) = 8
318710 munmap(0xf54be0dfd000, 63707)    = 0
318710 brk(NULL)                        = 0xc231b3239000
318710 brk(0xc231b3400000)              = 0xc231b3400000
318710 fstat(0, {st_mode=S_IFREG|0664, st_size=64, ...}) = 0
318710 read(0, "32 2 2 2 2 2 2\n100 5\n10\n1000 0.0"..., 4096) = 64
318710 fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
318710 write(1, "0.5\n", 4 <unfinished ...>
318709 <... read resumed>, "0.5\n", 4096) = 4
```

```
318710 <... write resumed>)             = 4
318709 write(1, "0.5\n", 4)              = 4
318710 write(1, "20\n", 3 <unfinished ...>
318709 read(4, "20\n", 4096)            = 3
318710 <... write resumed>)             = 3
318709 write(1, "20\n", 3 <unfinished ...>
318710 write(1, "10\n", 3 <unfinished ...>
318709 <... write resumed>)             = 3
318710 <... write resumed>)             = 3
318709 read(4, "10\n", 4096)            = 3
318710 write(1, "100000\n", 7 <unfinished ...>
318709 write(1, "10\n", 3 <unfinished ...>
318710 <... write resumed>)             = 7
318709 <... write resumed>)             = 3
318710 write(1, "500\n", 4 <unfinished ...>
318709 read(4 <unfinished ...>
318710 <... write resumed>)             = 4
318709 <... read resumed>, "100000\n500\n", 4096) = 11
318710 write(1, "0.05\n", 5 <unfinished ...>
318709 write(1, "100000\n500\n", 11 <unfinished ...>
318710 <... write resumed>)             = 5
318709 <... write resumed>)             = 11
318710 read(0 <unfinished ...>
318709 read(4 <unfinished ...>
318710 <... read resumed>, "", 4096)    = 0
318709 <... read resumed>, "0.05\n", 4096) = 5
318710 write(1, "0\n", 2 <unfinished ...>
318709 write(1, "0.05\n", 5)            = 5
318710 <... write resumed>)             = 2
318709 read(4 <unfinished ...>
318710 exit_group(0 <unfinished ...>
318709 <... read resumed>, "0\n", 4096) = 2
318709 write(1, "0\n", 2)               = 2
318710 <... exit_group resumed>)        = ?
318709 read(4, "", 4096)                = 0
318709 close(4)                         = 0
318709 wait4(318710 <unfinished ...>
318710 +++ exited with 0 +++
318709 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
↪  318710
318709 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=318710,
↪  si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
318709 exit_group(0)                    = ?
318709 +++ exited with 0 +++
```